APK静的解析と通信メタデータの統合による開発意図との照合支援システム

理論・実装・予備動作確認と評価計画

著者名・所属・学籍番号：提出時に記入

2026年10月29日改訂
研究用プロトタイプ 0.1.0 の技術資料

概要

AI支援や初学者によるAndroidアプリ開発では，開発者が想定していない権限宣言，第三者SDK，外部通信が含まれる可能性がある。しかし，APK内のURLや権限だけから通信の実行や必要性を判断することはできず，暗号化通信の宛先や送受信量だけから送信データの意味を確定することもできない。本資料では，開発者が明示した許可・禁止・未指定の条件を，APK静的解析と通信メタデータに照合するローカルシステムを記述する。提案方式は，判定，観測状態，アプリ帰属，本文可視性を独立に保持し，根拠と解析範囲を追跡可能にする。Python実装は静的解析，形式を固定したアプリ情報付きログおよびPCAPの取り込み，ルール照合，JSON・CSV・HTML出力を備える。予備確認では公開サンプルAPKに加え自作10 APKを解析し，静的な期待値103項目の一致を確認した。自動テスト65件と，ホスト内のHTTP・HTTPS・UDP参照サーバの通信試験も成功した。これらは開発確認であり，未知アプリでの精度評価ではない。実験者のAndroid Emulatorで取得したHTTP記録では，200応答，対象UIDの接続1行，参照サーバのPOST，ブラウザ前後対照と前後時計差を確認した。これは予備実測であり，インストールAPKの同定を含む受入試験の完了と未知アプリへの精度評価は今後の課題である。本資料の結果は実装の限定的な動作確認であり，漏えい検出性能やプライバシー保証を示すものではない。

キーワード：Android，APK，静的解析，通信メタデータ，開発意図，証拠追跡性

利用にあたって　本資料は卒業論文の方法・実装・評価計画の章に利用できる技術草稿である。大学の様式に合わせ，研究上の新規性と関連研究の調査，著者自身の実験，指導教員による確認を加えて完成させる必要がある。未実施の実験を結果として補わない。後掲の一次資料は仕様確認のための資料であり，既存研究全体のレビューの代替ではない。

目次

1 研究背景と問題設定 **4**
1.1 開発意図と実装のずれ 4
1.2 静的解析と動的観測の相補性 4
1.3 研究目的と主張の範囲 4

2 技術的基盤と既存ツールとの関係 **4**
2.1 APKと静的な証拠 4
2.2 Androidの通信設定 4
2.3 通信観測ツール 5
2.4 公式実装資料に基づく方式の比較 5

3 理論モデル **5**
3.1 入力と証拠の形式化 5
3.2 意図の明示性 6
3.3 部分情報に基づく判定 6
3.4 独立した観測軸 6
3.5 ホスト名照合と未知値 7
3.6 アプリ帰属 7
3.7 操作との時間的対応 7
3.8 未観測の条件 7

4 システム設計 **7**
4.1 構成とデータの流れ 7
4.2 静的解析手順 8
4.3 動的取得と操作管理 8
4.4 ログアダプタ 9
4.5 照合規則 9
4.6 証拠の管理と再現性 9

5 実装と利用方法 **10**
5.1 実装環境とモジュール 10
5.2 標準的な実行例 10
5.3 資源制限と入力保護 10
5.4 記録保護と共有版 11

6 予備的な動作確認 **11**
6.1 位置付け 11
6.2 公開サンプルAPK 11
6.3 自動テストと合成入力 12
6.4 未検証の項目 12

6.5 自作制御APKによる追加検証 12
6.6 参照サーバと集計系の確認 13
6.7 実験者のEmulatorによるHTTP初回観測 13
6.8 前後対照と時計測定を伴うHTTP観測 14

7 研究評価の計画 **15**
7.1 研究課題 15
7.2 二段階の評価 15
7.3 指標の定義 15
7.4 実験条件と比較方法 16
7.5 妥当性への脅威 16

8 制約と今後の課題 **16**

9 結論 **17**

A 卒業論文へ取り込む際の作業 **17**

1　研究背景と問題設定

1.1　開発意図と実装のずれ

Androidアプリは，自ら実装した処理だけでなく，ライブラリ，SDK，OSサービスとの連携を含む。たとえば「端末内で記録を保存する」という目的のアプリでも，ビルド設定や依存関係によって広告・解析用のコードや通信権限が含まれる可能性がある。AIによるコード生成の利用は，開発者がすべての依存や初期化経路を把握していない状況の一例となる。ただし，本研究ではAI生成アプリの方が危険であると事前に仮定しない。生成手段と不一致の発生頻度の関係は，条件を統制した別途の評価が必要である。

本研究でいう「開発意図」は，開発者が解析前に固定した構造化条件である。自然言語の目的文から必要な通信先を機械的に推定して禁止事項を生成すると，評価者の後付け解釈が混入し得る。したがって，目的文は説明として保存し，実際の照合には明示的なJSON規則を用いる。

1.2　静的解析と動的観測の相補性

静的解析は，実行しなかった経路を含むコードや宣言を調べられる一方，未使用のURLやSDKの残存コードも候補に含める。動的観測は実際に起きた通信の証拠を得られる一方，操作，待機時間，ネットワーク条件に依存し，観測されなかった経路を網羅できない。両方式は単独では結論できない点が異なるため，証拠を統合する意義がある。

1.3　研究目的と主張の範囲

目的は，開発意図と異なる可能性のある権限・SDK・通信先・通信タイミングを，根拠付きで提示することである。主張の対象は「照合支援」と「根拠の追跡性」であり，全アプリの完全解析，漏えいの自動断定，通信の必要性の自動決定は対象としない。許可ドメインへの接続が確認されても，送信内容や目的の妥当性は保証されない。

対象は，解析・実行の権限がある単一APKである。split APK・AAB，ネイティブコードの意味解析，動的ロードや高度な難読化への対処は初期実装の範囲外とする。

2　技術的基盤と既存ツールとの関係

2.1　APKと静的な証拠

APKはZIPコンテナであり，Manifest，DEX，リソース，署名関連ファイル等を含む。Manifestはパッケージ名，SDK条件，権限，コンポーネント，通信セキュリティ設定への参照を宣言する。DEXの文字列・クラス・命令参照は，URL候補やSDK特徴を探す資料となる。AndroguardはAPK・AXML・DEX等を扱う解析ライブラリであり，本実装では版を固定して利用する[1]。

2.2　Androidの通信設定

平文通信やCA信頼の扱いは，実行OS，target SDK，Manifest属性，Network Security Configに依存する[2]。Android API 24以降のNSCを利用する場合，Manifestの平文フラグだけから有効設定を決めない。ドメイン別設定には継承があり，ベース設定と例外を保存する必要がある。また，設定上の平文許可と実際にHTTP等を観測した事実は別の指摘である。

標準設定におけるユーザーCAの信頼はtarget SDKに依存する。設定画面にCAが存在することだけでは，対象アプリがそのCAを信頼するとはいえない。独自CAストアやピンニングは追加の制約となる。本システムのメタデータ観測では，CA導入やAPKの再署名を必須としない。

2.3 通信観測ツール

Emulatorのパケット取得は仮想ネットワークの通信を記録できる[3]が，通常のPCAPだけでは対象アプリのUIDが得られない。PCAPdroidはVPN方式のアプリ情報付き通信観測を提供する既存ツールである[4]。現実装は固定ソースの2.0.2形式に限ってCSVを直接取り込む。実験者のEmulatorで得た初回CSVの列形式と対象UIDの一致を確認したが，帰属精度の研究評価は未実施である。

mitmproxyによるHTTP/HTTPSの詳細取得は補助的な手段である。Android System ProxyとEmulator Proxyは役割が異なり，一部アプリはSystem Proxyを経由しない[5]。HTTP/3等の対応も利用モードに制約がある[6]。したがって，復号できなかったことと通信が発生しなかったことを区別しなければならない。

既存ツールが収集した証拠を開発意図の明示性と結び付け，不明・未観測・失敗を維持したまま根拠付きの結果に変換する点が，本システムの設計上の焦点である。研究上の新規性を主張する際は，意図ベースの監査，モバイルアプリのプライバシー分析，SDK検出，通信帰属に関する査読文献を追加調査し，比較する必要がある。

2.4 公式実装資料に基づく方式の比較

FlowDroidの公式資料は，Android/Javaにおけるsourceからsinkへの静的データフロー解析と，解析対象の定義・SDK JAR・タイムアウト等を説明している[8]。本システムはデータフローを追跡しておらず，同等の漏えい解析能力を主張しない。LibScoutは元ライブラリから作成したプロファイルでライブラリと版を識別し，難読化等を考慮する[9]。現実装の接頭辞ルールはそれより限定的であり，実SDK検出精度を別途評価する必要がある。

PCAPdroidの固定ソースを調べると，CSVはUID，package，通信量，時刻等を含む一方，表示用状態が複数の内部状態をまとめ，Info列もSNI/DNS/HTTP等の出典を保持しない[10]。本実装はその欠落を不明として保持する。ここで実読した資料は公式READMEとソースであり，査読論文の本文を実読した比較とは区別する。文献本文の一部はネットワーク制限で取得できず，関連研究レビューと新規性の確定は未完了である。

3 理論モデル

3.1 入力と証拠の形式化

APKを A，開発意図を I，操作シナリオを U，観測条件を C とする。静的候補集合と動的証拠集合を次のように表す。

$$S = f_{\text{static}}(A, R_s), \tag{1}$$

$$D = f_{\text{capture}}(A, U, C), \tag{2}$$

$$F = g(I, S, D, R_c), \tag{3}$$

ここで R_s は静的特徴ルール，R_c は照合ルールである。S は実行事実を意味しない。D は条件 C に依存するため，別の操作や待機条件で通信が生じないことを保証しない。

各証拠 e を，出典，場所，時刻，観測段階，帰属，本文可視性からなる組として保存する。

$$e = (\mathrm{id}, \mathrm{source}, \mathrm{locator}, t_0, t_1, o, a, v). \tag{4}$$

静的証拠では時刻は実行時刻を表さず，DEXの文字列番号や命令オフセット，Manifest内の宣言を所在とする。逆コンパイル結果の位置を原ソースの行番号とは扱わない。

3.2 意図の明示性

カテゴリ k の意図 I_k は，禁止，許可集合，未指定のいずれかを取る。

$$I_k \in \{\mathrm{deny_all},\ \mathrm{allowlist}(L_k),\ \mathrm{unspecified}\}, \quad L_k \neq \varnothing. \tag{5}$$

空の許可集合はdeny_allで表現する。allowlistと非空集合を要求し，deny_allと非空許可集合の併記を入力エラーとすることで，条件の意味を一意にする。未指定を暗黙の禁止へ変換しない。

3.3 部分情報に基づく判定

証拠の不足と条件の未指定は異なる。帰属や宛先が不明な場合，意図が明確でも比較できない。判定関数の基本形は次のとおりである。

$$J(I_k, e) = \begin{cases} \mathrm{needs_review} & I_k = \mathrm{unspecified}, \\ \mathrm{not_evaluable} & \text{照合に必要な証拠が不足}, \\ \mathrm{mismatch_candidate} & \text{明示条件と観測証拠が矛盾}, \\ \mathrm{consistent_with_observed_evidence} & \text{取得した証拠の範囲で整合}. \end{cases} \tag{6}$$

実装では帰属不明・取得失敗を先に除外する場合があるため，条件未指定であっても評価に必要な前提自体が成立しなければ判定不能を出力する。整合は，取得した項目への局所的な判断であり，アプリ全体の安全性を意味しない。

3.4 独立した観測軸

表1の軸を独立に保存する。単一の危険度スコアへ集約すると，「宣言だけの違い」と「対象アプリの実接続」の差，および未知情報が失われるためである。

表 1: 判定と観測情報の軸

軸	値・意味
verdict	不一致候補，観測範囲で整合，要確認，判定不能
observation	static_only，not_observed，connection_attempt，transport_established，tls_established，http_observed，capture_failed
attribution	target，other，related_candidate，unknown
content visibility	none，metadata_only，plaintext_available
run status	completed，partial，invalid，failed（現実装では主にcompleted/partial/failedを利用）

TCP接続試行，TCP成立，TLS成立，HTTP要求観測は異なる段階である。CONNECTはプロキシに対するトンネル作成要求であり，上流へのTLS成立や本文送信成功を意味しない。PCAPアダプタでは3ウェイハンドシェイクを確認した場合にTCP成立とし，単一セグメントのClientHelloからSNIが読めてもTLS成立へ昇格させない。

3.5 ホスト名照合と未知値

正規化関数 $N(h)$ はIDNのASCII変換，末尾ドットの除去，大小文字の統一，IPの標準表記化を行う。サブドメインの許可フラグを b とすると，照合は次で定義する。

$$M(h, d, b) = [N(h) = N(d)] \vee [b \wedge \neg \mathrm{IP}(d) \wedge N(h)\text{が}.N(d)\text{で終わる}]. \tag{7}$$

ドット境界が必要であり，example.comに対してbadexample.comは一致しない。許可要素はホスト，ポート集合，アプリケーションプロトコル集合を持つ。ポートのみからプロトコルを推定しない。プロトコルが制約されているのに値が不明なら判定不能とする。

同じIPを複数ドメインが共有することやDNSキャッシュがあることから，IPだけで許可ドメインの利用を肯定・否定できない。DNSからの対応付けが曖昧なら，ホスト名をnullとし理由を記録する。現PCAPアダプタはDNS応答からアプリ接続のホストを確定せず，直接見えるHTTP HostやSNIを利用する。

3.6 アプリ帰属

対象APKのpackageを p_A，そのインストールのUIDを u_A とする。ログのUID u_e が一致し，package情報と矛盾しない場合に対象候補とする。package名だけが一致してもUIDが不明ならunknownとする。再インストールでUIDが変わり得るため，u_A は実験ごとに取得する。

同じUIDを共有する構成，OSサービスへの委譲，外部ブラウザ経由の処理には限界がある。またSDKはアプリと同一UIDで動作し得るため，UID一致はSDK名への帰属を意味しない。別UIDの通信を単に無関係とせず，取得元の注記がある場合はrelated_candidateとする。

3.7 操作との時間的対応

ログ時刻 t_d は，測定した差分 Δ を用いてホスト基準に変換する。

$$t_h = t_d + \Delta, \qquad E(e) = \{u_i \mid t_i^{\mathrm{start}} \le t_h \le t_i^{\mathrm{end}},\ u_i\text{が成功}\}. \tag{8}$$

対応する操作IDは時間的な関連を表すのみで，因果関係を示さない。Home後区間は操作上の定義であり，OSの厳密なバックグラウンド実行状態と同一視しない。待機対照と反復を用いて代替説明を検討する必要がある。

3.8 未観測の条件

not_observedは，正常性が確認された観測系で，指定区間内に該当記録がなかった場合の限定的表現である。取得前後の対照不成立，欠落区間，操作失敗がある試行を「通信なし」の正解として集計しない。さらに，対照が成立しても観測ツールやアダプタの対応範囲外の通信は残る。

4 システム設計

4.1 構成とデータの流れ

入力検証器はスキーマ，APK形式，サイズ，矛盾を検査する。静的解析器は候補の抽出だけを担当し，禁止判定は照合器に分離する。観測アダプタは入力形式ごとの証拠を共通のフロー形式へ変換する。レポートはJSON正本から生成し，独立した判定軸を保存する。

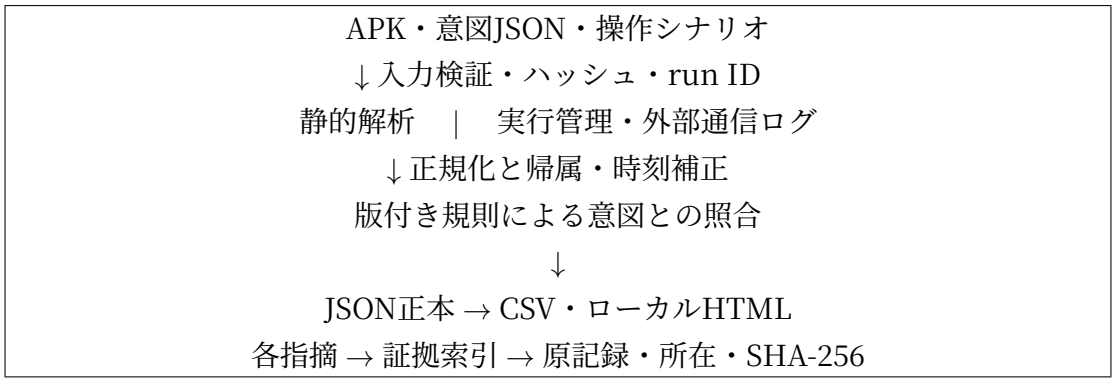


図 1: システムの処理構成。静的候補と動的観測は別の証拠として保持する。

4.2 静的解析手順

処理の概要を以下に示す。

1. APKのサイズ，ZIPメンバーの名前，重複，展開量，Manifest・DEXの有無を検査する。
2. 元APKを改変せず保存し，SHA-256を計算する。
3. AndroguardでManifestを復号し，基本情報，権限，コンポーネント，通信設定を取得する。
4. すべてのclasses系DEXを走査し，文字列，クラス特徴，登録済みAPIへのinvoke命令を抽出する。
5. 選択したリソースとARSC文字列を走査し，URL候補の元値，正規化結果，所在を保存する。
6. SDKルールのclass接頭辞とManifest文字列を照合し，単一種類と複数種類の特徴一致を区別する。
7. 処理済みファイル，対象外ファイル，警告，失敗を出力する。

現在のSDKルールはFirebase Analytics，Google Mobile Ads，Facebook App Events，Sentry Androidの4種である。接頭辞一致は候補検出であり，バージョン確定，実行，到達可能性，通信主体を証明しない。AndroidX等の一般ライブラリを一括して禁止SDKと扱わない。

4.3 動的取得と操作管理

実装したADB連携は，明示選択されたローカルEmulatorを対象に，元APKのインストール，UID取得，必要に応じたデータ初期化，パケット取得，UI操作，取得停止を管理する。launchとtapはresource-idによる到達確認を要求する。手動操作は担当者が完了を入力するまで成功扱いにしない。

Emulator consoleの取得前後でICMP対照を実行し，PCAP内の要求・応答を検査する。ただしこれは一般的な到達性と取得経路の確認であり，HTTP/HTTPS/UDPおよびアプリ別帰属の正解試験を代替しない。現クラウドではADB・Emulator・KVMが利用できず，本連携は未検証である。要件P0で実出力，API/ABI，時計，取得経路，UID帰属を検証してから固定構成として採用する。

4.4 ログアダプタ

app-flowは本実装で定義したJSONL契約である。時刻，宛先IP・ポート，TCP/UDP，観測段階，任意のホスト名と根拠，UID，package，送受信量を入力する。加えて，PCAPdroid 2.0.2の固定ソースに基づくCSVアダプタを実装した。原CSVの行を証拠に保存し，列ずれや未確認の版を拒否する。表示状態から接続成立やTLS成立へ昇格させず，Info列からホストの出典を推定しない。実験者から受領したEmulatorのエクスポートは列契約と一致した。後続のHTTP試行では前後時計差・区間ブラケット・前後対照を確認し，個別正規化まで実施した。インストールAPKの同定と正式runへの取り込みは残る。

PCAP/PCAPNGアダプタはクライアントIP側から通信方向を定める。60秒アイドルと新規SYNを区切りとしてフローをまとめるため，その件数は真の接続イベント数と一致する保証がない。TCP再構成やIP断片再構成は未実装であり，SNI・HTTP Hostの抽出範囲は限定される。PCAP単独のアプリ帰属はunknownとする。

同一ファイルの二重取り込みを拒否する。異なるログ同士の同一接続自動突合は行わず，取得元IDを維持する。二重計上を避けるため取得元を横断した単純合計は出さない。この点は完全なログ統合に向けた残課題である。

4.5 照合規則

表 2: 実装した照合の基本規則

ID	対象	判定の考え方
C01	通信禁止	対象アプリの接続試行を不一致候補とする。接続成立・本文送信とは区別。
C02	通信先許可リスト	帰属，ホスト，ポート，取得できるプロトコルを比較。不明をリスト外とみなさない。
C03	SDK	登録ルールで検出した候補を意図と照合。初期化・送信は断定しない。
C04	権限	Manifest宣言を比較。実際の権限行使やデータ取得を意味しない。
C05	平文	設定上の許可と実際に観測した平文プロトコルを別指摘にする。
C06	Home後通信	成功した操作区間内の対象通信を比較。因果やSDK起因を断定しない。
C07	不足と未指定	条件未指定は要確認，帰属・解析・観測の不足は判定不能として残す。

4.6 証拠の管理と再現性

各runにはAPKのハッシュ，入力条件，版，ホスト情報，操作時刻，取得条件，エラーを保存する。原記録，正規化結果，照合結果を分離し，指摘から証拠ID，原ファイル，レコード位置，ハッシュへ到達できるようにする。レポート生成時に原ファイルのハッシュを再確認する。

ハッシュは保存後の変更検出と入力同定のためのもので，第三者がログを作成した事実や真実性を証明する電子署名ではない。取得系の対照やオフラインUID情報の自己申告には，原記録の保管と実験者による検証が必要である。

依存ライブラリは間接依存を含め版とハッシュを固定する。静的解析の再現性比較では，run ID，時刻，所要時間を除く候補・所在・規則版を比較する。動的結果にはOSスケジュ

ーリング，ネットワーク応答，初回同意，キャッシュ等が影響するため完全一致を要求しない。

5　実装と利用方法

5.1　実装環境とモジュール

検証した環境はLinux x86_64，Python 3.12.14である。主要ライブラリはAndroguard 4.1.3，jsonschema 4.25.1，dpkt 1.9.8，defusedxml 0.7.1である。プロジェクト版は0.1.0，ルール版は2026-10-27.1である。

表 3: Pythonモジュールと責務

モジュール	責務
core / contracts	時刻・ハッシュ・JSON入出力・スキーマ・正規化
static / worker	APK解析と子プロセスの隔離・資源制限
android	ADB事前確認・操作・取得停止（Android実証は未実施）
capture	app-flowとPCAPの正規化・帰属・証拠索引
compare	意図との照合，独立した判定軸
report / share	CSV・HTML，原本を含まない共有用要約
cli / demo	コマンド操作，合成デモ

5.2　標準的な実行例

```
bash tools/setup.sh
source .venv/bin/activate
apk-audit static --apk app.apk --out runs/app
apk-audit compare --run runs/app --intent intent.json
apk-audit report --run runs/app
```

外部ログはimportコマンドで追加し，compareとreportを再実行する。Android実行には別途検証したAVDを必要とする。利用手順，JSON例，終了コード，出力形式の詳細はリポジトリのREADMEとcapture-format資料に記載する。

5.3　資源制限と入力保護

初期値はAPK上限200 MiB，ZIP内の宣言展開量1 GiB，静的解析10分，静的ワーカのアドレス空間3 GiB，動的1試行10分とする。これらは防御的な設定値であり，このサイズまでの性能を実証した数値ではない。リソース単体が8 MiBを超える場合は対象外とし，未処理範囲に記録する。

ZIPメンバーの絶対パス，親ディレクトリ参照，危険な区切り，重複を検査し，入力名のまま展開先パスを作らない。JSON文字列をshell命令として実行せず，引数配列を用いる。HTMLはエスケープし，外部資源に依存しない。CSVでは先頭の式記号を無害化する。

子プロセスと資源制限は被害範囲を抑えるが，解析ライブラリの脆弱性を完全に防ぐサンドボックスを実装したものではない。未知APKを扱う研究環境自体も隔離する必要がある。

5.4　記録保護と共有版

研究者用の原本にはトークンや個人情報を含み得るURL・ログが残る。共有版は許可された判定項目だけを抽出し，目的文，ホスト，パス，UID，原本ログ，証拠リンクを省く。これを完全に匿名化した原本や根拠付きレポートとは呼ばず，「証拠を含まない共有用要約」と区別する。

6　予備的な動作確認

6.1　位置付け

本節は実装の動作確認であり，未知アプリへの有効性の研究評価ではない。正解集合を確定した精度実験と，自動テストの成功件数は分ける。クラウドにはAndroid実行機能がなく，自動ADB連携の実行は未検証である。一方，実験者のMacとEmulatorから受領したHTTP初回記録を本節で別途報告する。

6.2　公開サンプルAPK

Androguard公式リポジトリのコミット

`581f7a3251545a97f66ee92b674dd810ee380a00`

にある`tests/data/APK/hello-world.apk`を動作確認に使用した。ファイルサイズは1,722,314 bytes，SHA-256は

`f427a0ebe0bca97b9acf6cd2a2a01c37a7d3762841810fc54a7191ec637330b2`

である。APKを実行・改変せずに解析した。結果を表4に示す。

表 4: 単一公開APKに対する静的解析の予備確認

項目	出力
package	`de.rhab.helloworld`
versionName	1.0
minSdk / targetSdk	21 / 25
DEXファイル	1
権限宣言	0
URL候補（所在ごとの候補）	202
登録対象APIへのinvoke参照	41
登録ルールによるSDK候補	0
初回解析時間（親プロセス計測）	約3.68秒
ワーカの最大RSS（Linux計測）	157,316 KiB

URL数はユニークな接続先数でも実通信数でもない。同じURLが異なる所在にあれば複数候補となる。SDK候補0件は4種の登録ルールへの一致がなかったことだけを示し，SDKの不存在を意味しない。権限宣言0件についても，既知正解との網羅的照合を実施して精度を算出したものではない。

このAPKに旅行記録アプリ用の架空の意図を適用した処理確認では，平文設定の指摘1件，URL候補の要確認202件，動的未実施の判定不能1件を出力した。対象APKの本来の開発意図を評価した結果ではなく，レポートの処理例に限定する。

解析時間とRSSは単一の実行例であり，性能の平均・上限・優位性を主張しない。反復一致の検査では生成時刻や所要時間を除いて静的出力を比較する。

6.3 自動テストと合成入力

自動テストでは，ドメイン・IP・IDNの正規化，ドット境界，矛盾する意図，帰属不明，CONNECT-only，重複取り込み，破損APK，ZIPパストラバーサル，HTML・CSV注入，証拠改変，部分失敗を検査する。追加のテストはNSC継承，target SDKの平文既定，解析不能，タイムアウト保存，非有限JSON値の拒否を扱う。実行件数とログの最終記録は`docs/validation.md`に記載する。

合成PCAPではTCPハンドシェイクとHTTP Hostを確認する。合成デモでは，対象UIDの接続とHome後区間を意図に照合して不一致候補を出力する。これらはプログラムの分岐を検証する素材であり，端末上でHTTP・HTTPS・UDP通信を取得した証拠ではない。

6.4 未検証の項目

Android API・ABI・AVDイメージを固定した実験，別アプリを独立に操作する同時通信対照，VPNによる経路影響，端末からのサーバ到達性の条件間比較，遅延送信とクラッシュ，未知アプリでの静的ルール精度は未検証である。HTTPではPCAPdroid CSV，時計差・前後対照・アプリ応答を個別照合したが，APK同定，欠落・失敗条件，HTTPS・UDP・無通信を含む取得系の受入試験は未完了である。したがって要件書のMVPが完成したとは主張しない。

6.5 自作制御APKによる追加検証

通信なし，HTTP，HTTPS，UDP，位置取得のみ，位置取得とHTTPS，30秒遅延，Activity非表示後，意図的クラッシュ，複数DEXの10素材を作成した。Android Platform 35，Build Tools 35.0.0，ECJ 3.39.0を用いてビルドし，研究用debug鍵で署名して公式apksignerで検証した。初回ビルド時点ではAndroid上で未実行であった。その後，HTTP素材のUIを修正した版1.1を実験者がEmulatorで操作し，後述する初回観測を得た。他の動的ケースは未検証である。

期待値はソースとManifestの生成仕様から定めた。パッケージ名は独立したaapt2の出力とも照合した。APK原本ハッシュ，権限，DEX，URL所在，通信設定，署名情報等の合計103項目が10素材すべてで一致した。結果を表5に示す。同じ開発過程で素材・期待値・解析器を作っており，未見データに対する精度推定ではない。

表 5: 自作開発用素材の静的確認。Checksは期待値との一致項目数。

Case	Checks	URLs	API refs	Time (s)
offline	10/10	2	0	0.92
http	10/10	3	12	0.72
https	10/10	2	12	0.47
udp	10/10	2	6	0.52
location_only	10/10	2	2	0.52
location_https	10/10	2	16	0.37
delayed	10/10	3	12	0.53
after_home	10/10	3	12	0.52
crash	10/10	2	0	0.37
static_multidex	13/13	3	1	0.37

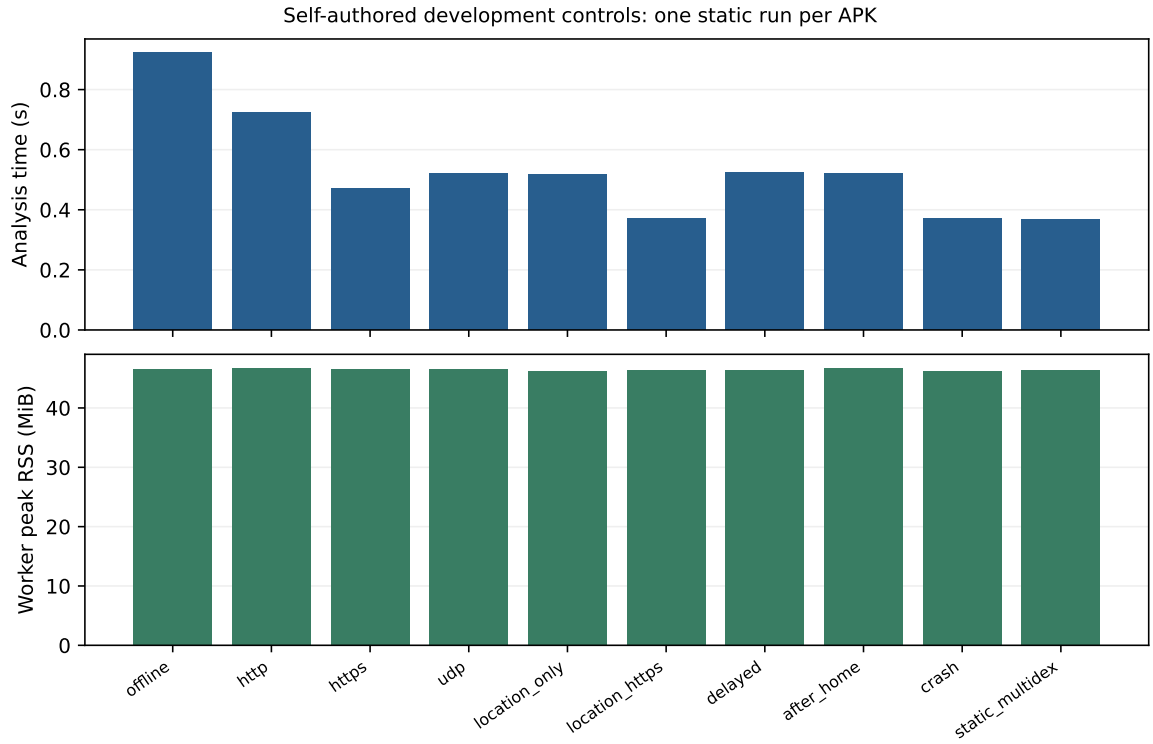


図 2: 各制御APKの1回の解析時間とワーカ最大RSS。母集団の性能・上限を示すものではない。

複数DEX素材では，classes2.dexに置いた未使用URLとAPI参照の所在，NSCのドメイン別継承を確認した。Sentryの接頭辞を持つクラスは自作の模擬クラスであり，本物のSDKではない。これは特徴ルールへの一致を確認する素材であり，実SDKの検出精度で正例と数えてはならない。接頭辞一致だけでは実SDKの存在を確定できないことも示している。

6.6 参照サーバと集計系の確認

HTTP・HTTPS・UDPの研究用参照サーバを実装した。本文や認証ヘッダを保存せず，実験ID，本文byte数・ハッシュ，接続ID，受信イベントを記録する。ホストのループバック上でHTTP要求2件，HTTPS要求2件，UDPデータグラム1件を実際に送受信した。HTTPとHTTPSは各1本の接続で2要求ずつを処理し，TCP接続総数2とHTTP系要求総数4を区別できた。TLS検証は有効で，試験用証明書を当該クライアントだけに明示的に信頼させた。この結果はAndroid側の取得・帰属の検証を代替しない。

自動テストは追加後65件が成功した。3方式の証拠選択，CSV列ずれの拒否，実際のループバックHTTPS，棄権を陰性へ変換しない集計器を含む。条件付き適合率・再現率に加え，全正解陽性に対する検出割合と評価可能率を出力する。分母0は未定義とし，実アプリの正解ラベルが未取得の現段階では，未知アプリへの精度値は報告しない。

6.7 実験者のEmulatorによるHTTP初回観測

2026年10月29日，実験者のMacとAndroid Emulatorで取得したPCAPdroid CSV，対象アプリと取得ツールのpackage情報，参照サーバ記録を受領した。

対象は`org.example.apkaudit.http`版1.1（versionCode 2），取得ツールはPCAPdroid 2.0.2である。package情報のappIdとUser 0のインストール状態から対象UIDを10207と確認した。結果を表6に示す。これは手動による初回の観測経路確認であり，自動ADB連携

や未知アプリへの精度評価を検証したものではない。

表 6: HTTP初回観測の記述集計

項目	観測結果
CSVデータ行数	6
対象UIDの行数	1（UID 10207）
対象行の宛先	TCP，`10.0.2.2:8765`
対象行の送信／受信量	514／466 bytes（取得ツールの計数値）
別UIDの行数	5（Google Play開発者サービス，UID 10133）
サーバの対象POST	1件，`run_id=ui-check-001`
サーバの要求本文サイズ	18 bytes

CSVは固定ソースで確認した17列形式と一致した。対象行は原CSVの6行目にあり，開始時刻は15:15:08.747 JST，終了時刻は15:15:08.751 JSTであった。サーバのPOST受信は原JSONLの20行目，15:15:09.031456 JSTに記録され，本文ハッシュは制御アプリの`audit=ui-check-001`と一致した。通信量514／466 bytesと本文18 bytesは測定対象が異なり，同一量として比較しない。

UID・package・宛先と近接時刻は対応に整合するが，CSVにはrun IDやサーバの接続IDがないため，異なる取得元の同一接続対応は候補に留める。未補正のサーバ受信時刻とCSV開始時刻の差は0.284456秒である。時計差が未測定なので，この差を遅延や時計補正値として解釈しない。CSVの表示状態だけからHTTP応答200や本文内容を確定せず，POST受信はサーバ記録を根拠とする。

取得開始・停止，前後対照，操作時刻，アプリ画面・logcat，AVD情報，インストール済みAPKのハッシュは未提出である。別UIDの背景通信は，独立して操作した対照アプリの正解とは区別する。正式な動的importの必須条件を推測で補わず，適合率・再現率・帰属精度は算出しない。原本4点のハッシュ，行位置，未測定項目は`docs/research/http-pilot-2026-10-29.md`と保存済み個別解析に記録した。

6.8 前後対照と時計測定を伴うHTTP観測

次の試行`http-004`では，アプリの画面，継続logcat，PCAPdroid CSV，参照サーバ，前後の時計測定とブラウザ対照を受領した。表7に結果を示す。対象アプリ版は1.1，PCAPdroidは2.0.2，対象UIDは10207である。

表 7: HTTP試行`http-004`の記述集計

項目	結果
アプリの操作・応答	action_started 1件，HTTP 200応答1件
CSV行数	全21行，対象1行，別UID 20行
対象行の宛先	TCP，`10.0.2.2:8765`
対象行の送信／受信量	506／466 bytes
サーバの対象POST	1件，実験ID一致，要求本文14 bytes
ブラウザ対照	前後とも画面のevent IDとサーバGETが一致
サーバの接続／HTTP要求	TCP受付5件，GET 4件，POST 1件
時計測定	前後各5回，原サンプルからsummaryを再計算

対象行は原CSVの12行目，POSTは原JSONLの9行目にある。アプリのlogcatと画面の双方で`http_response=200`を確認し，POST本文ハッシュは`audit=http-004`と一致した。前後のブラウザ画面の応答IDは，それぞれ原JSONLの3行目と12行目のGETに一致した。別UIDの20行はChrome 10行，Google Play開発者サービス8行，Messages 2行であり，対象通信へ合算しない。

Mac時刻に合わせる補正差の範囲は，取得前で0.261942338〜0.285341005秒，取得後で0.260753207〜0.286018374秒だった。両者には共通範囲があり，差が一定という仮定下で中点約0.273642秒を用いた個別正規化を行った。前後中点の変化約−0.256 msはバッチの幅より小さい。途中の時計変更や観測区間全体の誤差上限を実証したものではない。アプリ記録上の操作から応答記録までは約0.406秒で，通信だけのRTTではない。

後のブラウザ対照は，補正モデル上で操作開始約14.13秒後に実行された。実験者からその時点で操作したと確認されている。元手順の30秒待機案との差を記録し，今回は実際の待機条件によるHTTP成功確認とする。固定30秒待機の結果とは扱わない。CSVには操作約36.145秒後の新規接続もあり，後対照の時点と取得停止時点は異なる。

今回のCSVは実アダプタで21行を検査・正規化できたが，インストールAPKのハッシュが未提出であり，正式runへのimportと3方式比較は保留した。前後対照・時刻・操作の証拠がそろったHTTPの予備実測として扱い，未知アプリへの帰属精度や適合率・再現率は算出しない。原本16点のハッシュと詳細は`docs/research/http-004-results.md`および保存済み個別解析に記録した。

7　研究評価の計画

7.1　研究課題

1. 権限・SDK・URL候補を，固定した正解集合に対してどの程度抽出できるか。
2. 正解の接続イベントを観測でき，対象アプリへ正しく帰属できるか。
3. 明示した意図との不一致候補を，誤断定を抑えて提示できるか。
4. 静的のみ，動的のみ，統合の条件で，得られる証拠と手作業量がどう変わるか。

7.2　二段階の評価

第一段階では，ソースが既知の正解アプリと研究用サーバを用いる。通信なし，HTTP，HTTPS，位置取得のみ，位置取得とHTTPS送信を基本とし，SDK，遅延通信，Home後通信，別アプリ同時通信，プロキシ迂回を追加する。各素材の実装と仕様を人手で確認し，サーバ側の接続ID・受信記録・端末側記録を対応付ける。

第二段階ではAI生成・初学者作成アプリへ適用する。生成タスク，プロンプト，モデル，日付，修正回数，ビルド結果を保存する。意図JSONは解析結果を見る前に確定する。元プロンプトにない制約を後から追加して生成AIの違反と呼ばず，追加監査条件として別に報告する。

7.3　指標の定義

正解陽性と指摘単位を固定し，TP，FP，FNを定める。

$$\mathrm{Precision} = \frac{TP}{TP + FP}, \qquad \mathrm{Recall} = \frac{TP}{TP + FN}, \tag{9}$$

$$\mathrm{AttributionAccuracy} = \frac{\text{正しく帰属した接続数}}{\text{帰属を付けた接続数}}, \quad \mathrm{UnknownRate} = \frac{\text{帰属不明の接続数}}{\text{評価対象接続数}}. \tag{10}$$

分母0の場合は0と置かず未定義として報告する。権限・SDK・URL，静的候補と実接続はそれぞれ別集計とする。判定不能を陰性に置き換えると精度が過大評価されるため，その割合を独立に示す。

動的観測率の分母は観測区間内の正解接続イベント数とする。HTTP要求は接続再利用や多重化があるため，要求数をTCP接続数の正解に置き換えない。実行コストとして解析時間，操作時間，手動作業量，失敗率，最大メモリを計測する。

7.4 実験条件と比較方法

まず1つのAVD構成を選び，API，ABI，イメージ版，Googleサービス，ホスト仕様，ツール版を固定する。初期提案値は操作後30秒，Home後120秒，3反復である。これは遅延通信を網羅する保証ではない。追加の待機時間，CA・proxy・VPNの変更は別実験条件として保存する。

条件間比較では，同じAPK，同じ意図，同じシナリオと初期状態を用い，静的のみ・動的のみ・統合の差を調べる。必要に応じて実行順を入れ替え，ネットワークや初回状態の影響を検討する。小標本の割合は信頼区間や実数を併記し，母集団への一般化を抑える。

表 8: 今後埋める評価結果の表（現時点では未測定）

評価項目	静的のみ	動的のみ	統合
不一致候補の適合率	未測定	未測定	未測定
不一致候補の再現率	未測定	未測定	未測定
対象アプリ帰属精度	対象外	未測定	未測定
判定不能率	未測定	未測定	未測定
手動確認時間	未測定	未測定	未測定

7.5 妥当性への脅威

内的妥当性　初回同意，アプリデータ，権限，キャッシュ，時刻ずれ，サーバ応答が差の原因になり得る。反復と待機対照，条件の保存で検討する。操作直後の通信という時間関係だけで因果を断定しない。

構成概念妥当性　URL文字列数は通信数の代理にならない。許可先への通信は安全の代理にならない。SDK候補とSDKによる送信も別の概念である。評価単位を事前に固定する。

外的妥当性　単一API/ABI，少数の自作アプリ，4種のSDKルールから他アプリ全般へ一般化できない。難読化，動的ロード，ネイティブ実装，ログインが必要な機能を限界に残す。

評価者の偏り　解析器の出力だけで正解表を作らない。ルール調整用と評価用素材を分ける。独立評価者を確保できない場合は，同一人物が正解表と評価を作成したことを明記する。

8 制約と今後の課題

最優先の課題は要件P0の実証である。アプリ情報付き取得ツールの採用版を決め，HTTP・HTTPS・UDP・通信なしの正解ケースを別アプリ同時通信とともに確認する。HTTPの前後時計差・取得区間・前後対照の個別照合を出発点とし，APK同定を含む正式な取り込み，ADB自動操作とクラッシュ記録を検証する。

静的解析では今回の複数DEXとNSC継承の開発確認に加え，リソース構成差や実SDKを含む独立した正解APKを整備する。SDKルールの追加は評価用の答えだけに合わせず，出典・

版・検出限界を管理する。動的解析ではTCP再構成，DNSメタデータの独立保存，複数取得元の同一接続対応，logcatによるクラッシュ検出が課題である。

HTTPSの詳細取得は補助プロファイルとして分離し，元APK・標準設定でのメタデータ観測と比較する。システムCA導入にはAVD種別等の制約があり[7]，ピンニングや独自CAを必ず回避できるとはしない。復号の成功率だけをシステムの価値にせず，取得できない情報を明示しながら次の確認箇所を示す支援を評価する。

9 結論

本資料では，APKの静的候補と通信メタデータを，明示的な開発意図に照合する方式を定式化し，Pythonプロトタイプの実装と限定的な動作確認を示した。設計上の要点は，未指定と禁止，候補と実通信，帰属不明と対象通信，未観測と不存在を区別し，判定から証拠への追跡経路を保存することである。静的解析とログ処理に加え，実験者のEmulatorでのHTTP初回観測を得た段階にある。HTTPの前後時計差・対照・応答を確認した一方，APK同定と他ケースを含む観測系の受入試験，独立した正解表に基づく研究評価が残る。

A 卒業論文へ取り込む際の作業

著者情報と大学様式を反映する。関連研究の比較を査読文献に基づいて追加する。採用AVDと収集器の版を固定し，受入試験を実施する。評価計画を事前に確定し，実験ログと正解表を保存した上で，未測定の表を実測に置き換える。本文中の予備確認と合成デモを本評価と混同しない。研究支援ツールの利用に関する所属機関の規定がある場合は，その規定に従って執筆過程を説明する。

参考文献

[1] Androguard project, “Getting Started” and official repository. https://androguard.readthedocs.io/en/latest/intro/gettingstarted.html; https://github.com/androguard/androguard. 実装では4.1.3を使用。

[2] Android Developers, “Network security configuration.” https://developer.android.com/privacy-and-security/security-config.

[3] Android Developers, “Start the emulator from the command line.” https://developer.android.com/studio/run/emulator-commandline.

[4] Emanuele Faranda et al., “PCAPdroid,” official repository. https://github.com/emanuele-f/PCAPdroid.

[5] Android Developers, “Set up Proxy with the Android Emulator.” https://developer.android.com/studio/run/emulator-networking-proxy.

[6] mitmproxy project, “Protocols.” https://docs.mitmproxy.org/stable/concepts/protocols/.

[7] mitmproxy project, “Install System CA Certificate on Android Emulator.” https://docs.mitmproxy.org/stable/howto/install-system-trusted-ca-android/.

[8] FlowDroid project, “FlowDroid Data Flow Analysis Tool,” official README, commit 9bf10349ec6fdceebbcfda2d261a1f23b9e2e439. https://github.com/

secure-software-engineering/FlowDroid. 公式実装資料を確認。査読論文本文の引用ではない。

[9] LibScout project, official README, commit 8c0bd2c147e818841fc695eecb1246e38458435f. https://github.com/reddr/LibScout. 公式実装資料を確認。

[10] PCAPdroid project, “ConnectionsAdapter.dumpConnectionsCsv,” “ConnectionDescriptor,” “Utils,” and “strings.xml,” commit 66f7b3514540c20add8655a74dfee93d00a9563b, versionName 2.0.2. https://github.com/emanuele-f/PCAPdroid. 固定ソースのCSV契約を確認。初回Emulator出力との列一致を確認したが，帰属精度評価は未実施。

参照URLは添付要件書の一次資料を引き継いだ。オンライン資料は更新され得るため，卒論提出時に利用API・採用版に対応する記述とアクセス日を再確認すること。公開APKの取得元コミットとハッシュは第6節に固定した。